



Locks: Solusi untuk Critical Section

Week 11: Lock Abstraction, Mutex, dan Atomic Instructions

informatika-itera.github.io/os

Program Studi Teknik Informatika
Institut Teknologi Sumatera

2026

OUTLINE

Peta materi pertemuan ini

1. **Lock Abstraction**
2. **Hardware Foundation: Instruksi Atomik**
3. **Kebenaran dan Keadilan Lock**
4. **Deadlock**
5. **Rangkuman**

Capaian Pembelajaran

Tujuan setelah pertemuan selesai

1. Menjelaskan apa itu lock dan mengapa kita membutuhkannya.
2. Menggunakan `pthread_mutex` untuk melindungi *critical section*.
3. Mengenali pola lock/unlock yang benar dan yang berbahaya.
4. Menjelaskan bagaimana instruksi atomik (`test-and-set`, `compare-and-swap`) menjadi fondasi lock.
5. Membandingkan spinlock dan mutex dari sisi kebenaran dan performa.
6. Mengevaluasi lock menggunakan tiga syarat critical section dari minggu lalu.

Fokus Hari Ini

Kita mulai dari **Bagian 1: Lock Abstraction** — intuisi, API, dan pola dasar.

Lock Abstraction

Ingat Masalah Minggu Lalu?

Dua thread menghitung bersama — hasilnya malah salah

Kode yang terlihat benar, tapi hasilnya tidak bisa diprediksi

```
int counter = 0; // dihitung bersama dua thread

void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++)
        counter++; // <-- di sinilah masalahnya
    return NULL;
}
```

Dua thread masing-masing menambah counter 1 juta kali. Harusnya **2.000.000** — tapi bisa lebih kecil.

Pertanyaan untuk kita hari ini

Bagaimana agar counter++ tidak bisa diganggu thread lain **di tengah-tengah**?

Ide Dasarnya Sederhana: Kunci Pintu

Bayangkan ruangan yang hanya boleh diisi satu orang



Bayangkan ada **ruangan khusus** yang hanya boleh dimasuki **satu orang** pada satu waktu.

- Thread 1 masuk → **kunci pintu dari dalam**
- Thread 2 datang → **tidak bisa masuk**, harus tunggu di luar
- Thread 1 selesai → **buka kunci**
- Baru Thread 2 boleh masuk

Inilah yang disebut **lock**

Lock = mekanisme yang memastikan hanya **satu thread** yang boleh mengakses data penting pada satu waktu.

Lock Punya Dua Perintah Saja

Cukup dua kata untuk mengatur giliran antar thread

acquire — “Saya mau masuk”

- Thread minta izin masuk ke kode sensitif
- Belum ada yang di dalam → **langsung masuk**
- Sudah ada yang di dalam → **tunggu dulu**

release — “Saya sudah keluar”

- Thread selesai, membuka kunci
- Thread lain yang menunggu **boleh masuk sekarang**

Aturan paling penting

Pada satu waktu,
hanya SATU thread
yang boleh memegang lock.

Urutan wajib

acquire



kerjakan data sensitif



release

Di C, Lock Disebut `mutex`

Empat langkah untuk memakai mutex — hafal urutannya

```
// 1: Siapkan variabel mutex
pthread_mutex_t lock;

// 2: Aktifkan sebelum dipakai
pthread_mutex_init(&lock, NULL);

// 3a: Kunci sebelum masuk
pthread_mutex_lock(&lock);
// ... kerjakan data sensitif ...

// 3b: Buka kunci setelah selesai
pthread_mutex_unlock(&lock);

// 4: Bersihkan di akhir program
pthread_mutex_destroy(&lock);
```

Empat langkah

1. Buat variabel mutex
2. `init` — hidupkan
3. `lock` → kerja → `unlock`
4. `destroy` — bersihkan

Aturan tidak boleh dilanggar

Setiap lock **wajib** punya pasangan `unlock`.

Sekarang Kita Perbaiki counter++

Tambahkan lock dan unlock di sekitar baris yang bermasalah

Versi lama — bermasalah

```
int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++)
        counter++; // thread lain bisa
                    // menyela di sini!
    return NULL;
}
```

Versi baru — aman

```
int counter = 0;
pthread_mutex_t lock; // tambahan

void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&lock);
        counter++; // aman sekarang
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

Kenapa sekarang aman?

Saat T1 di antara lock dan unlock, T2 **tidak bisa masuk**. LOAD-ADD-STORE tidak bisa disela. Hasil selalu tepat **2.000.000**.

Lihat Apa yang Terjadi di Garis Waktu

Thread 2 sabar menunggu sampai Thread 1 benar-benar selesai



Perhatikan: bagian biru T1 dan bagian hijau T2 **tidak pernah terjadi di waktu yang sama**. Itulah yang kita inginkan — tidak ada yang saling menyela.

Hati-Hati: Jangan Lupa Membuka Kunci

Kesalahan kecil ini bisa membuat program berhenti selamanya

✓ Pola yang benar

```
pthread_mutex_lock(&lock);  
counter++; // kerja  
pthread_mutex_unlock(&lock);  
// kunci sudah dibuka, aman
```

Setiap lock punya pasangan unlock.
Thread lain tidak akan kehabisan giliran.

× Pola berbahaya

```
pthread_mutex_lock(&lock);  
if (ada_error) return;  
// BAHAYA: unlock terlewat!  
counter++;  
pthread_mutex_unlock(&lock);
```

Kalau return dijalankan, unlock terlewat.
Thread lain **menunggu selamanya**.

Cara mengingat

Bayangkan kamu mengunci pintu kamar mandi lalu keluar lewat jendela. Pintu tetap terkunci — orang berikutnya tidak bisa masuk!

Saat Menunggu, Thread Melakukan Apa?

Ada dua cara menunggu — dan keduanya punya konsekuensi berbeda

Cara 1: Tidur dulu (Mutex)

1. Thread minta lock
2. Lock belum bebas → OS **tidurkan thread**
3. Thread berhenti, tidak pakai CPU
4. Lock dilepas → OS **bangunkan thread**

Ambil nomor antrean, duduk sampai dipanggil.

Cara 2: Terus Cek (Spinlock)

1. Thread minta lock
2. Lock belum bebas → thread **loop terus**
3. `while (terkunci) { cek lagi }`
4. Lock dilepas → keluar dari loop

Berdiri di kasir dan terus tanya “sudah bisa belum?”

Mana yang lebih baik?

Mutex hemat CPU (thread tidur). Spinlock lebih cepat jika tunggu **sangat singkat**, tapi boros CPU kalau lama.

Sekarang Kita Coba Sendiri

Jalankan dua program, lihat perbedaannya dengan mata kepala sendiri

Program 1: Tanpa mutex

```
cd 4_handson_code/week11
gcc -Wall -pthread -O0 \
    -o race 01_race.c
./race
```

Jalankan **3 kali**. Catat hasilnya.
Apakah selalu sama?

Program 2: Dengan mutex

```
gcc -Wall -pthread -O0 \
    -o mutex 02_mutex.c
./mutex
```

Jalankan **3 kali**. Catat hasilnya.
Apakah selalu sama?

Catat tiga hal ini

1. `race`: apakah hasilnya berubah-ubah? Berapa jauh dari 2.000.000?
2. `mutex`: apakah hasilnya selalu 2.000.000?
3. Mana yang lebih cepat, dan mengapa?

Checkpoint: Paham Sampai Mana?

Empat pertanyaan — jawab tanpa melihat slide sebelumnya

1. Lock punya dua perintah utama. Sebutkan keduanya dan jelaskan apa yang terjadi kalau lock sedang dipakai thread lain.
2. Sebutkan empat fungsi `pthread_mutex` secara berurutan — dari deklarasi hingga selesai dipakai.
3. Kenapa kode seperti ini berbahaya? `lock()` → if error return → ... → `unlock()`
4. Apa bedanya mutex dan spinlock dari sisi **apa yang dilakukan thread** saat menunggu?

Jawaban

1. `acquire` (kunci) dan `release` (buka). Kalau sudah dipakai, thread lain harus tunggu.
2. `init` → `lock` → *kerja* → `unlock` → `destroy`.
3. Jalur `return` melewati `unlock` — kunci tidak pernah dibuka, thread lain menunggu selamanya.
4. Mutex: thread tidur, tidak pakai CPU. Spinlock: thread terus cek dalam loop, CPU tetap terpakai.

Hardware Foundation

Mengapa Software Saja Tidak Cukup?

Percobaan Dulu: Buat Lock Sendiri dari Kode

Minggu lalu kita coba — hasilnya masih bisa salah. Kenapa?

Ide awal: pakai variabel “bendera”

```
int flag = 0; // 0 = bebas, 1 = terkunci

void lock() {
    while (flag == 1) { } // tunggu
    flag = 1; // ambil kunci
}

void unlock() {
    flag = 0;
}
```

Kelihatannya masuk akal

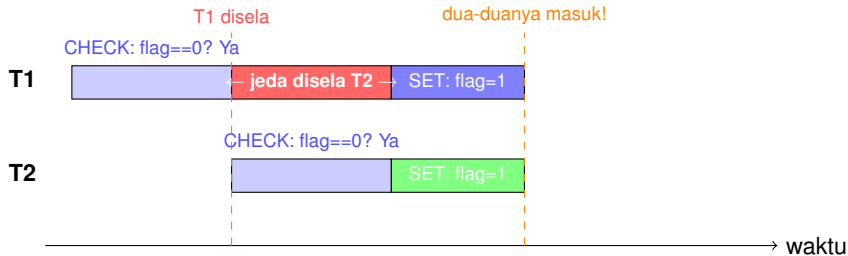
- Cek dulu: apakah flag bebas?
- Kalau bebas → set flag = 1 (kunci)
- Kalau sudah dikunci → tunggu

Pertanyaan

Ada yang salah di sini.
Bisakah kamu menemukan lubangnya?

Masalahnya: Ada Jeda Berbahaya di Antara Dua Langkah

CHECK dan SET adalah dua instruksi terpisah — dan bisa disela!



T1 sudah cek flag = 0, tapi belum sempat set flag = 1.
T2 masuk di celah itu — cek juga 0 — keduanya **lolos bersama**. Kunci gagal.

Solusinya: Satu Langkah yang Tidak Bisa Disela

Kita butuh CHECK dan SET terjadi sekaligus — tidak ada jeda

Masalah software-only

1. CHECK (cek flag)
2. — *bisa disela di sini* —
3. SET (ubah flag)

Dua langkah terpisah = ada peluang masalah.

Yang kita butuhkan

1. CHECK + SET **sekaligus**
2. Tidak ada yang bisa masuk di tengah
3. Dilakukan dalam **satu instruksi CPU**

Satu langkah = tidak ada celah.

Inilah yang dimaksud **Atomik**

Kata *atom* berarti “tidak bisa dibagi”.

Instruksi atomik = instruksi yang dijalankan **utuh atau tidak sama sekali**.

CPU menjamin tidak ada thread lain yang bisa menyela di tengahnya.

Test-and-Set: Instruksi Atomik Pertama

Satu instruksi, dua hal sekaligus — baca nilai lama, lalu langsung ganti

Cara kerja (pseudocode)

```
// Ini SATU instruksi CPU
// tidak bisa disela
int TestAndSet(int *flag, int new) {
    int old = *flag; // baca nilai lama
    *flag = new; // langsung tulis baru
    return old; // kembalikan nilai lama
}
```

Pakai untuk lock

```
// lock berhasil jika nilai lama = 0
// lock gagal jika nilai lama = 1
while (TestAndSet(&flag, 1) == 1)
    ; // masih terkunci, tunggu
// sekarang kita yang pegang lock
```

Analogi kasir

Bayangkan mesin tiket yang hanya punya satu tombol:

*“Ambil nomor antrean saat ini **dan** langsung ganti ke nomor berikutnya.”*

Dua hal itu terjadi bersamaan — tidak ada yang bisa menyerobot di antaranya.

Kuncinya

Nilai lama dikembalikan **sebelum** nilai baru ditulis.

Jika dapat 0 → kita berhasil kunci.

Jika dapat 1 → orang lain duluan.

Bangun Spinlock dengan Test-and-Set

Ini adalah spinlock paling sederhana yang bisa dibuat di C

```
#include <pthread.h>

int flag = 0; // 0=bebas, 1=terkunci

void spin_lock(int *f) {
    // __sync_lock_test_and_set adalah
    // versi C dari instruksi Test-and-Set
    while (__sync_lock_test_and_set(f, 1) == 1)
        ; // putar terus sampai dapat 0
}

void spin_unlock(int *f) {
    // tulis 0 dengan cara atomik
    __sync_lock_release(f);
}

// Pemakaian:
spin_lock(&flag);
counter++; // aman sekarang
spin_unlock(&flag);
```

Cara baca kodenya

- `__sync_lock_test_and_set` = Test-and-Set bawaan GCC
- Kalau dapat 1 → orang lain masih di dalam, terus putar
- Kalau dapat 0 → kita berhasil masuk, tulis 1 sekaligus

Ingat

Spinlock **sibukkan CPU** selama menunggu. Cocok hanya untuk tunggu sangat singkat.

Compare-and-Swap: Instruksi Atomik yang Lebih Pintar

Tukar nilai hanya jika kondisinya terpenuhi — kalau tidak, biarkan

Cara kerja (pseudocode)

```
// Ini SATU instruksi CPU
int CompareAndSwap(int *addr,
                   int expected,
                   int new) {
    if (*addr == expected) {
        *addr = new; // tukar!
        return 1; // berhasil
    }
    return 0; // tidak ditukar
}
```

Bedanya dengan Test-and-Set

Test-and-Set		Compare-and-Swap
Selalu tukar		Tukar <i>hanya jika</i> nilai cocok
Kembalikan lama	nilai	Kembalikan sukses/gagal
Sederhana		Lebih fleksibel

Analogi

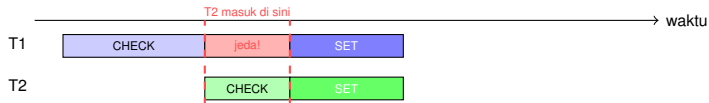
Seperti memasukkan koin ke loket:

“Buka hanya jika kunci cocok — jika tidak, tidak berubah apa-apa.”

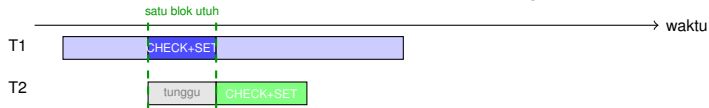
Lihat Bedanya: Atomik vs Non-Atomik

Instruksi atomik menutup “jendela bahaya” yang ada di software-only

Non-Atomik (software-only) — Ada jendela bahaya



Atomik (Test-and-Set) — Jendela tertutup



Coba Sendiri: Spinlock vs Mutex

Dua cara lock — satu sibuk menunggu, satu tidur menunggu

Program 3: Spinlock

```
gcc -Wall -pthread -O0 \  
    -o spin 03_spinlock.c  
./spin
```

Jalankan **3 kali**. Catat hasilnya.
Apakah selalu 2.000.000?

Bandingkan dua lock

```
time ./mutex # mutex  
time ./spin # spinlock
```

Mana yang lebih cepat?

Yang perlu diobservasi

1. Keduanya harus menghasilkan 2.000.000 — kebenaran sama
2. Kecepatan bisa berbeda — lihat output time
3. Coba pantau CPU: `top` saat spinlock berjalan

Diskusi

Spinlock mungkin lebih cepat karena tidak ada overhead sleep/wake. Tapi CPU-nya 100%.
Mutex lebih hemat — tapi ada biaya bangun-tidur.
Mana yang cocok untuk situasi apa?

Checkpoint 2: Hardware Foundation

Empat pertanyaan — jawab tanpa melihat slide sebelumnya

1. Kenapa flag biasa (variabel `int`) tidak cukup untuk membuat lock yang benar meskipun logikanya kelihatan masuk akal?
2. Apa yang dimaksud instruksi **atomik**? Berikan satu contoh hal yang dijamin oleh instruksi atomik.
3. Jelaskan cara kerja Test-and-Set dalam satu atau dua kalimat. Nilai apa yang dikembalikan jika kunci **sudah diambil** orang lain?
4. Apa perbedaan utama antara Test-and-Set dan Compare-and-Swap?

Jawaban

1. CHECK dan SET adalah dua instruksi terpisah — ada jeda di antaranya. Thread lain bisa masuk di celah itu.
2. Instruksi yang dijalankan utuh, tidak bisa disela di tengah. Contoh: Test-and-Set menjamin baca dan tulis terjadi sekaligus.
3. Test-and-Set membaca nilai lama lalu langsung menulis nilai baru dalam satu instruksi. Jika kunci sudah diambil (`flag=1`), ia kembalikan 1.
4. Test-and-Set selalu menulis nilai baru. Compare-and-Swap hanya menulis jika nilai saat ini sama dengan yang diharapkan — lebih selektif.

Kebenaran dan Keadilan Lock

Apakah Lock Kita Sudah Benar-Benar Aman?

Ingat Tiga Syarat dari Minggu Lalu?

Lock harus memenuhi ketiganya — mari kita periksa satu per satu

Tiga syarat critical section

1. Mutual Exclusion

Hanya satu thread di dalam pada satu waktu.

2. Progress

Kalau tidak ada yang di dalam, thread yang mau masuk tidak boleh diblokir selamanya.

3. Bounded Waiting

Setiap thread yang menunggu pasti mendapat giliran — tidak boleh menunggu selamanya.

Pertanyaan yang kita jawab hari ini

Apakah `pthread_mutex`
dan **spinlock** memenuhi
ketiga syarat ini?

Spoiler

Syarat 1 terpenuhi. Syarat 3 **belum tentu** —
tergantung implementasi.

Syarat 1: Mutual Exclusion — Terpenuhi

Ini adalah tujuan utama lock — dan keduanya berhasil

✓ Mutex

- Thread minta lock → OS periksa
- Kalau sudah ada yang di dalam → thread **tidur**, tidak bisa masuk
- Hanya satu thread aktif di critical section

✓ Spinlock

- Thread minta lock → Test-and-Set atomik
- Kalau sudah diambil → thread **loop**, tidak bisa masuk
- Hanya satu thread berhasil ambil lock (nilai lama = 0)

Kenapa keduanya lulus?

Karena instruksi di bawahnya **atomik**.

Mutex pakai syscall OS yang atomic.

Spinlock pakai Test-and-Set atomic.

Tidak ada “celah” untuk dua thread masuk bersamaan.

Mutual exclusion = tujuan utama lock. Kedua jenis lock **lulus syarat ini**.

Syarat 3: Bounded Waiting — Belum Tentu Terpenuhi

Apakah setiap thread yang menunggu pasti dapat giliran?

✓ Mutex (biasanya)

OS menyimpan **antrian** thread yang menunggu. Saat lock dilepas, OS pilih thread berikutnya dari antrian.

Seperti antrian kasir — siapa datang duluan, dilayani duluan.

Tapi bergantung pada implementasi OS.

× Spinlock — tidak ada jaminan

Spinlock **tidak punya antrian**.

Saat flag = 0, semua thread yang spinning berlomba pakai Test-and-Set.

Thread mana yang menang? **Tidak ada yang tahu**.

Thread tertentu bisa kalah terus — **starvation**.

Apa itu Starvation?

Thread *kelaparan* = thread menunggu sangat lama (atau selamanya) karena thread lain terus-terusan mendapat giliran duluan.

Checkpoint 3: Kebenaran Lock

Tiga pertanyaan — jawab tanpa melihat slide sebelumnya

1. Sebutkan tiga syarat yang harus dipenuhi oleh sebuah mekanisme critical section.
2. Mengapa mutex dan spinlock keduanya memenuhi syarat **Mutual Exclusion**? Apa yang menjamin hal itu?
3. Apa yang dimaksud *starvation*? Mengapa spinlock lebih rentan terhadap starvation dibandingkan mutex?

Jawaban

1. Mutual Exclusion, Progress, Bounded Waiting.
2. Keduanya menggunakan instruksi atomik di bawahnya (syscall OS untuk mutex, Test-and-Set untuk spinlock) sehingga tidak ada celah untuk dua thread masuk bersamaan.
3. Starvation = thread menunggu sangat lama karena selalu kalah bersaing. Spinlock tidak punya antrian — saat lock bebas, semua thread spinning berlomba tanpa urutan, sehingga thread tertentu bisa selalu kalah.

Deadlock

Ketika Tidak Ada yang Bisa Maju

Deadlock: Dua Thread Saling Tunggu

Ini lebih berbahaya dari lupa unlock — tidak ada yang bisa maju

Skenario

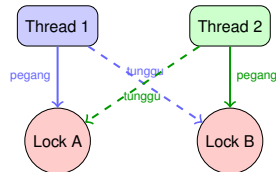
Thread 1	Thread 2
lock(A)	lock(B)
<i>(pegang A)</i>	<i>(pegang B)</i>
lock(B) ...	lock(A) ...
<i>tunggu B</i>	<i>tunggu A</i>

Hasilnya

T1 tunggu B yang dipegang T2.

T2 tunggu A yang dipegang T1.

Keduanya menunggu selamanya.



Solusi singkat

Selalu ambil lock dengan **urutan yang sama** di semua thread.

Empat Kondisi yang Menyebabkan Deadlock

Keempatnya harus terpenuhi sekaligus — cukup satu yang dilanggar, deadlock tidak terjadi

- C1 Mutual Exclusion:** resource hanya boleh dipakai satu thread pada satu waktu.
- C2 Hold and Wait:** thread sudah memegang resource, masih minta resource lain.
- C3 No Preemption:** resource tidak bisa diambil paksa dari thread yang sudah pegang.
- C4 Circular Wait:** ada chain melingkar di mana T1 tunggu T2, T2 tunggu T3, ..., Tn tunggu T1.

Implikasi

Jika kita bisa **melanggar salah satu kondisi**, deadlock **tidak akan terjadi**.
Itulah dasar strategi pencegahan deadlock.

Resource Allocation Graph: Gambar Deadlock

Visualisasi yang membantu mendeteksi apakah deadlock sudah terjadi

Komponen

- **Node Thread** ($T_1, T_2, \dots$): digambar sebagai persegi
- **Node Resource** ($R_1, R_2, \dots$): digambar sebagai persegi dengan titik di dalam
- **Panah “hold”**: $T \rightarrow R$ = thread menahan resource
- **Panah “request”**: $R \rightarrow T$ = thread minta resource

Deadlock terdeteksi

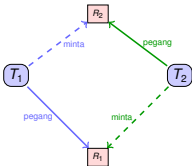
Jika ada **cycle** di graf,
dan setiap resource di cycle
hanya punya **satu instance**.

Deadlock terjadi.

Tidak deadlock

Cycle tapi ada resource dengan
multiple instances → mungkin
tidak deadlock, perlu cek lebih lanjut.

Deadlock: $T_1 \rightarrow R_1 \rightarrow T_2 \rightarrow R_2 \rightarrow T_1$



Cara Mencegah Deadlock

Langgar salah satu dari empat kondisi

Kondisi	Cara Melanggar	Implikasi
Mutual Exclusion	Buat resource tidak eksklusif	Tidak selalu bisa
Hold and Wait	Minta semua resource sekaligus	Boros, efisien turun
No Preemption	Ambil paksa resource	Bisa rusak state
Circular Wait	Urutan lock konsisten	Praktis

Strategi paling praktis

Urutan lock yang konsisten.

Jika semua thread mengambil lock A dulu baru B, maka **circular wait tidak akan pernah terbentuk.**

Checkpoint: Deadlock

Tiga pertanyaan — jawab tanpa melihat slide sebelumnya

1. Jelaskan empat kondisi yang harus terpenuhi agar deadlock terjadi.
2. Kapan kita bisa yakin deadlock terjadi hanya dengan melihat Resource Allocation Graph?
3. Apa strategi paling praktis untuk mencegah deadlock? Jelaskan alasannya.

Jawaban

1. Mutual exclusion, hold and wait, no preemption, circular wait. Keempatnya harus ada bersamaan.
2. Jika ada cycle dan setiap resource dalam cycle hanya punya satu instance.
3. Urutan lock konsisten — melanggar circular wait tanpa biaya tinggi.

Rangkuman Week 11

Apa yang Sudah Kita Pelajari

Rangkuman

Lock, Atomic Instructions, dan Deadlock

Lock

- Lock punya dua operasi: acquire (lock) dan release (unlock)
- `pthread_mutex`: sleep-based lock, hemat CPU
- Spinlock: busy-wait lock, lebih cepat untuk tunggu singkat

Hardware Foundation

- Software-only lock gagal karena CHECK dan SET terpisah
- Instruksi atomik: Test-and-Set, Compare-and-Swap
- Atomic = satu instruksi yang tidak bisa disela

Deadlock